1 Tymbolica

`symbolica` exposes Symbolica's exact algebra engine to Typst through a WebAssembly plugin. Expressions and matrices are passed around as opaque byte payloads; use `to-typst` for document output and `canonical` for portable text.

1.1 Setup

```
#import "@local/tymbolica:0.1.0": *

#let expr = math($(x + 1)^2$)
#to-typst(expand(expr))
```

For a source checkout, import the local library directly:

```
#import "lib.typ": *
```

1.2 Worked API Examples

Tidy evaluates each example below and places its rendered output beside the source. Lines beginning with >>> provide hidden setup shared only within that example.

1.2.1 Engine, Parsing, and Symbols

```
#let sym = init(namespace: "physics")
#let svar = sym.var

#let x = var("x")
#let y = var("y")
#let a-wild = wild("a")
#let rhs-only = wild("fresh")
#let expr = math($(x + 1)^2 + y$)
#let literal = atom("z")
#let namespaced = add(svar("x"), x)

expr: #to-typst(expr)\
literal: #to-typst(literal)\
wildcards: #raw(canonical(a-wild)) and #raw(canonical(rhs-only))\
namespaces: #raw(canonical(namespaced, namespaces: true))
```

expr: $y + (1 + x)^2$
literal: z
wildcards: `a_` and `fresh_`
namespaces: `typst::x+physics::x`

1.2.2 Parse Tree Inspection

```
#array-tree($(x + 1)^2$)
```

```
(
  strong(body: raw(text: "pow")),
  (
    styled(child: raw(text: "base"), ..),
    (
      strong(body: raw(text: "()")),
      (
        styled(child: raw(text: "expr"), ..),
        (
          strong(body: raw(text: "add")),
          equation(body: [x]),
          equation(body: [1]),
        ),
      ),
    ),
  ),
  (
    styled(child: raw(text: "exp"), ..),
    equation(body: [2]),
  ),
)
```

1.2.3 Rendering

```
#let expr = math($(x + 1)^2 + y$)

canonical: #raw(canonical(expr))\
to-typst-source: #raw(to-typst-source(expr))\
to-typst: #to-typst(expr)\
to-latex: #raw(to-latex(expr))
```

canonical: y+(1+x)^2
to-typst-source: y+(1+x)^2
to-typst: $y + (1 + x)^2$
to-latex: y+\left(1+x\right)^{2}

1.2.4 Algebra and Calculus

```
#let p = math($x^2 + 2 x + 1$)
#let q = math($(x + 1) (y + 2)$)
#let builtin = init(namespace: "symbolica")
#let bmath = builtin.math
#let bvar = builtin.var
#let bseries = builtin.series
#let bto-typst = builtin.to-typst
#let bx = bvar("x")
#let ser = bseries(bmath($cos(x)/(x + 1)$), bx, 0, 3)
#let integration = integrate-with-steps(p, x)

simplify: #to-typst(simplify(p))\
expand: #to-typst(expand(q))\
factor: #to-typst(factor(p))\
derivative: #to-typst(derivative(q, x))\
integral: #to-typst(integration.result)\
integration steps: #integration.steps.map(to-typst).join[, ]\
series: #bto-typst(ser)
```

simplify: $1 + 2x + x^2$
expand: $2 + 2x + xy + y$
factor: $(1 + x)^2$
derivative: $2 + y$
integral: $\frac{1}{3}(3x + 3x^2 + x^3)$
integration steps: $\frac{1}{3}(3x + 3x^2 + x^3)$
series: $1 - x + \frac{1}{2}x^2 - \frac{1}{2}x^3$

1.2.5 Scalar Constructors and Arithmetic

```
#let arith = add(
  mul(2, x, y),
  neg(z),
  sub(x, y),
  div(1, x),
  pow(x, 3),
)
#to-typst(arith)
```

$x + 2xy - y - z + x^3 + \frac{1}{x}$

1.2.6 Replacement

```
#let src = math($f(x, y) + x$)
#let swapped = replace(src, math($f("a_", "b_")$), math($g("b_", "a_")$))
#let r1 = rule(math($f("a_")$), math($h("a_")$))
#let r2 = rule(x, z)
#let multiple = replace-multiple(math($f(x) + x$), (r1, r2))
#let wild-replaced = replace-wildcards(math($k("a_")$), ((wild("a"), math($x + 1$)),))

replace: #to-typst(swapped)\
replace-multiple: #to-typst(multiple)\
replace-wildcards: #to-typst(wild-replaced)
```

replace: $x + g(y, x)$
replace-multiple: $z + h(x)$
replace-wildcards: $k(1 + x)$

1.2.7 Evaluation and Solving

```
#let value = evaluate(math($x^2 + y$), values: ((x, 2.0), (y, 3.0)))
#let complex = evaluate(math($x^2$), values: ((x, (re: 2.0, im: 1.0)),))
#let many = evaluate-many((math($x + y$), math($x y$)), (x, y), ((1, 2), (3, 4)))
#let grid = evaluate-grid(math($x^2 + y$), (x, y), (domain(-1, 1, samples: 3), domain(0, 1, samples: 2)))
#let exact = solve-linear((math($2 x + y - 5$), math($x - y - 1$)), (x, y))
#let nonlinear = solve-system((math($x + y$), math($y^2 - 2$)), (x, y))
#let root = nsolve(math($x^2 - 2$), x, 1.0)
#let roots = nsolve-system((math($x^2 + y - 3$), math($x - y$)), (x, y), (1.0, 1.0))

value: #repr(value)\
complex: #repr(complex)\
evaluate-many: #repr(many)\
evaluate-grid shape: #repr(grid.shape); first: #repr(grid.values.first())\
solve-linear: #exact.map(to-typst).join[, ]\
solve-system: #nonlinear.map(row => row.map(to-typst).join[, ]).join[; ]\
nsolve: #repr(root)\
nsolve-system: #repr(roots)
```

```
value: (re: 7.0, im: 0.0)
complex: (re: 3.0, im: 4.0)
evaluate-many: (
  ((re: 3.0, im: 0.0), (re: 2.0, im: 0.0)),
  ((re: 7.0, im: 0.0), (re: 12.0, im: 0.0)),
)
evaluate-grid shape: (3, 2) first: ((re: 1.0, im: 0.0),)
solve-linear: 2, 1
solve-system: √2, −√2; −√2, √2
nsolve: 1.4142135623746899
nsolve-system: (1.3027756377319948, 1.3027756377319948)
```

1.2.8 Matrix Construction and Rendering

```
#let A = matrix($mat(2, 1; 1, -1)$)
#let B = matrix(((1, 2), (3, 4)))
#let b = vec((5, 1))
#let I = identity(2)
#let D = eye((1, 2))

A: #to-typst(A)\
B: #to-typst(B)\
b: #to-typst(b)\
I: #to-typst(I)\
D: #to-typst(D)
```

A: $\begin{pmatrix} 2 & 1 \\ 1 & -1 \end{pmatrix}$
B: $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$
b: $\begin{pmatrix} 5 \\ 1 \end{pmatrix}$
I: $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
D: $\begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$

1.2.9 Matrix Operations

```
#let solved = matrix-solve(A, b)
#let any = matrix-solve-any(A, b)
#let reduced = row-reduce(B)
#let aug = augment(A, B)
#let parts = split-col(aug, 2)

add: #to-typst(matrix-add(A, B))\
sub: #to-typst(matrix-sub(B, A))\
mul: #to-typst(matrix-mul(A, I))\
```

```
scalar mul: #to-typst(matrix-mul(A, 3))\
scalar div: #to-typst(matrix-div-scalar(B, 2))\
transpose: #to-typst(transpose(A))\
det: #to-typst(det(A))\
inv: #to-typst(inv(A))\
solve: #to-typst(solved)\
solve-any: #to-typst(any)\
row-reduce: rank #reduced.rank, #to-typst(reduced.matrix)\
augment: #to-typst(aug)\
split-col: #to-typst(parts.at(0)) | #to-typst(parts.at(1))
```

add: $\begin{pmatrix} 3 & 3 \\ 4 & 3 \end{pmatrix}$
sub: $\begin{pmatrix} -1 & 1 \\ 2 & 5 \end{pmatrix}$
mul: $\begin{pmatrix} 2 & 1 \\ 1 & -1 \end{pmatrix}$
scalar mul: $\begin{pmatrix} 6 & 3 \\ 3 & -3 \end{pmatrix}$
scalar div: $\begin{pmatrix} \frac{1}{2} & 1 \\ \frac{3}{2} & 2 \end{pmatrix}$
transpose: $\begin{pmatrix} 2 & 1 \\ 1 & -1 \end{pmatrix}$
det: -3
inv: $\begin{pmatrix} \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & -\frac{2}{3} \end{pmatrix}$
solve: $\begin{pmatrix} 2 \\ 1 \end{pmatrix}$
solve-any: $\begin{pmatrix} 2 \\ 1 \end{pmatrix}$
row-reduce: rank 2, $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
augment: $\begin{pmatrix} 2 & 1 & 1 & 2 \\ 1 & -1 & 3 & 4 \end{pmatrix}$
split-col: $\begin{pmatrix} 2 & 1 \\ 1 & -1 \end{pmatrix}$ | $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$

1.2.10 Matrix Entries and Polynomial Content

```
#let P = matrix(((mul(2, x), mul(4, x)), (mul(6, x), mul(8, x))))

primitive-part: #to-typst(primitive-part(P))\
content: #to-typst(content(P))\
matrix-at: #to-typst(matrix-at(A, 0, 1))\
matrix-shape: #repr(matrix-shape(A))
```

primitive-part: $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$
content: $2x$
matrix-at: 1
matrix-shape: (2, 2)

1.3 API Reference

The reference below is generated from the doc comments in `lib.typ` with `tidy`. Its examples use a preamble equivalent to importing the package with `: *`.

1.4 symbolica

- add()
- array-tree()
- atom()
- augment()
- canonical()
- content()
- derivative()
- det()
- div()
- domain()
- evaluate()
- evaluate-grid()
- evaluate-many()
- expand()

- eye()
- factor()
- identity()
- init()
- integrate()
- integrate-with-steps()
- inv()
- math()
- matrix()
- matrix-add()
- matrix-at()
- matrix-div-scalar()
- matrix-mul()
- matrix-shape()
- matrix-solve()
- matrix-solve-any()
- matrix-sub()
- mul()
- neg()
- nsolve()
- nsolve-system()
- pow()
- primitive-part()
- replace()
- replace-multiple()
- replace-wildcards()
- row-reduce()
- rule()
- series()
- simplify()
- solve-linear()
- solve-system()
- split-col()
- sub()
- to-latex()
- to-typst()
- to-typst-source()
- transpose()
- var()
- vec()
- wild()

1.4.1 add

Add one or more expression payloads or Typst values.

```
#to-typst(add("x", 1, "y"))
```

$$1 + x + y$$

1.4.1.1 Parameters

`add(..terms) -> bytes`

1.4.2 array-tree

Render the Parsely tree used for parsing a math expression.

```
#array-tree($(x + 1)^2$)
```

```
(
  strong(body: raw(text: "pow")),
  (
    styled(child: raw(text: "base"), ..),
    (
      strong(body: raw(text: "()")),
      (
        styled(child: raw(text: "expr"), ..),
        (
          strong(body: raw(text: "add")),
          equation(body: [x]),
          equation(body: [1]),
        ),
      ),
    ),
  ),
  (
    styled(child: raw(text: "exp"), ..),
    equation(body: [2]),
  ),
)
```

1.4.2.1 Parameters

```
array-tree(
  eqn,
  grammar
) -> content
```

1.4.3 atom

Convert a Typst value or math expression into an atom payload.

```
#to-typst(atom("x"))
```

$$x$$

1.4.3.1 Parameters

```
atom(value) -> bytes
```

1.4.4 augment

Horizontally augment two matrices.

```
#to-typst(augment(identity(2), matrix(((1, 2), (3, 4)))))
```

$$\begin{pmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & 3 & 4 \end{pmatrix}$$

1.4.4.1 Parameters

```
augment(
  lhs,
  rhs
) -> bytes
```

1.4.5 canonical

Render an atom or matrix payload as Symbolica text.

```
#raw(canonical(math($x + 1$)))
```

1+x

1.4.5.1 Parameters

```
canonical(
  expr,
  namespaces
) -> str
```

1.4.6 content

Compute the content of a matrix as an atom payload.

```
#let x = var("x")
#let P = matrix(((mul(2, x), mul(4, x)),
(mul(6, x), mul(8, x))))
#to-typst(content(P))
```

$2x$

1.4.6.1 Parameters

```
content(matrix) -> bytes
```

1.4.7 derivative

Differentiate `expr` with respect to `var`.

```
#let x = var("x")
#to-typst(derivative(math($(x + 1)^2$), x))
```

$2(1+x)$

1.4.7.1 Parameters

```
derivative(
  expr,
  var
) -> bytes
```

1.4.8 det

Compute the determinant of a matrix as an atom payload.

```
#to-typst(det(matrix(((1, 2), (3, 4)))))
```

-2

1.4.8.1 Parameters

```
det(matrix) -> bytes
```

1.4.9 div

Divide two expression payloads or Typst values.

```
#to-typst(div(1, "x"))
```

$\frac{1}{x}$

1.4.9.1 Parameters

```
div(
  lhs,
  rhs
) -> bytes
```

1.4.10 domain

Construct a real interval sampled by `evaluate-grid`.

```
#repr(domain(-1, 1, samples: 3))
```

```
(min: -1.0, max: 1.0, samples: 3)
```

1.4.10.1 Parameters

```
domain(
  min,
  max,
  samples
) -> dictionary
```

1.4.11 evaluate

Evaluate an expression with optional real or complex values.

```
#let x = var("x")
#let y = var("y")
#repr(evaluate(math($x^2 + y$), values: ((x,
2.0), (y, 3.0))))
```

```
(re: 7.0, im: 0.0)
```

1.4.11.1 Parameters

```
evaluate(
  expr,
  values
) -> dictionary
```

1.4.12 evaluate-grid

Evaluate expressions over the Cartesian product of real domains in one Wasm call.

Returns `(shape, points, values)`. `points` contains real coordinates; complex `values` use `(re, im)` dictionaries. Both are flattened in row-major order, with the last domain varying fastest.

```
#let x = var("x")
#let y = var("y")
#let grid = evaluate-grid(math($x^2 + y$),
(x, y), (domain(-1, 1, samples: 3), domain(0,
1, samples: 2)))
shape: #repr(grid.shape); values:
#repr(grid.values)
```

```
shape: (3, 2) values: (
  ((re: 1.0, im: 0.0),),
  ((re: 2.0, im: 0.0),),
  ((re: 0.0, im: 0.0),),
  ((re: 1.0, im: 0.0),),
  ((re: 1.0, im: 0.0),),
  ((re: 2.0, im: 0.0),),
)
```

1.4.12.1 Parameters

```
evaluate-grid(
  expressions,
  variables,
  domains
) -> dictionary
```

1.4.13 evaluate-many

Evaluate one or more expressions at explicit parameter points in one Wasm call.

Each returned row corresponds to one input point, and each value is a `(re: float, im: float)` dictionary.

```
#let x = var("x")
#let y = var("y")
#let rows = evaluate-many((math($x + y$), math($x y$)), (x, y), ((1, 2), (3, 4)))
#repr(rows)
```

```
(
  ((re: 3.0, im: 0.0), (re: 2.0, im: 0.0)),
  ((re: 7.0, im: 0.0), (re: 12.0, im: 0.0)),
)
```

1.4.13.1 Parameters

```
evaluate-many(
  expressions,
  variables,
  points
) -> array
```

1.4.14 expand

Expand products and powers through Symbolica's polynomial expansion.

```
#to-typst(expand(math($(x + 1)^2$)))
```

$$1 + 2x + x^2$$

1.4.14.1 Parameters

```
expand(expr) -> bytes
```

1.4.15 eye

Create a diagonal matrix payload from diagonal entries.

```
#to-typst(eye((1, 2)))
```

$$\begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$$

1.4.15.1 Parameters

```
eye(diag) -> bytes
```

1.4.16 factor

Factor an expression payload.

```
#to-typst(factor(math($x^2 + 2 x + 1$)))
```

$$(1 + x)^2$$

1.4.16.1 Parameters

```
factor(expr) -> bytes
```

1.4.17 identity

Create an `n` by `n` identity matrix payload.

```
#to-typst(identity(2))
```

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

1.4.17.1 Parameters

```
identity(n) -> bytes
```

1.4.18 init

Create a Symbolica engine record backed by the WebAssembly plugin.

The returned dictionary contains the full API. The top-level functions below are convenience wrappers around `init()` with default settings.

```
#let sym = init(namespace: "physics")
#let v = sym.var
#let render = sym.canonical
#raw(render(v("x"), namespaces: true))
```

```
physics::x
```

1.4.18.1 Parameters

```
init(
  namespace: str,
  source: str,
  grammar: dictionary
) -> dictionary
```

1.4.18.1.1 namespace `str`

Default namespace used for symbols parsed from Typst math or strings.

Default: `"typst"`

1.4.18.1.2 source `str`

WebAssembly plugin path passed to Typst's `plugin` constructor.

Default: `"tymbolica.wasm"`

1.4.18.1.3 grammar `dictionary`

Parsely grammar used by `math`.

Default: `_default_grammar`

1.4.19 integrate

Integrate a polynomial exactly with respect to `var`.

1.4.19.1 Parameters

```
integrate(
  expr,
  var
) -> bytes
```

1.4.20 integrate-with-steps

Integrate a polynomial and expose the additive terms produced by the integration algorithm. Returns `(result: bytes, steps: array)`.

1.4.20.1 Parameters

```
integrate-with-steps(
  expr,
  var
) -> dictionary
```

1.4.21 inv

Compute the inverse of a matrix payload.

```
#to-typst(inv(matrix(((1, 2), (3, 4)))))
```

$$\begin{pmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{pmatrix}$$

1.4.21.1 Parameters

```
inv(matrix) -> bytes
```

1.4.22 math

Parse Typst math content into an opaque Symbolica atom payload.

```
#let expr = math($x + 1$)
#to-typst(expr)
```

$$1 + x$$

1.4.22.1 Parameters

```
math(
  eqn: content,
  grammar: dictionary none,
  namespace: str none
) -> bytes
```

1.4.22.1.1 eqn `content`

Math content, normally written as `$...$`.

1.4.22.1.2 grammar `dictionary` or `none`

Optional Parsely grammar override.

Default: none

1.4.22.1.3 namespace `str` or `none`

Optional namespace override for parsed symbols.

Default: none

1.4.23 matrix

Convert Typst math `mat(...)`, `vec(...)`, or nested arrays into a matrix payload.

```
#to-typst(matrix($mat(1, 2; 3, 4)$))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

1.4.23.1 Parameters

```
matrix(value) -> bytes
```

1.4.24 matrix-add

Add two matrix payloads.

```
#to-typst(matrix-add(matrix(((1, 2), (3,
4))), identity(2)))
```

$$\begin{pmatrix} 2 & 2 \\ 3 & 5 \end{pmatrix}$$

1.4.24.1 Parameters

```
matrix-add(
  lhs,
  rhs
) -> bytes
```

1.4.25 matrix-at

Read a matrix entry as an atom payload using zero-based indices.

```
#let A = matrix(((1, 2), (3, 4)))
#to-typst(matrix-at(A, 0, 1))
```

2

1.4.25.1 Parameters

```
matrix-at(
  matrix,
  row,
  col
) -> bytes
```

1.4.26 matrix-div-scalar

Divide a matrix by a scalar expression.

```
#to-typst(matrix-div-scalar(matrix(((2, 4),
(6, 8))), 2))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

1.4.26.1 Parameters

```
matrix-div-scalar(
  lhs,
  rhs
) -> bytes
```

1.4.27 matrix-mul

Multiply matrices, or multiply a matrix by a scalar expression.

```
#to-typst(matrix-mul(matrix(((1, 2), (3,
4))), identity(2)))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

1.4.27.1 Parameters

```
matrix-mul(
  lhs,
  rhs
) -> bytes
```

1.4.28 matrix-shape

Return the matrix shape as `(rows, columns)`.

```
#repr(matrix-shape(matrix(((1, 2), (3, 4)))))
```

(2, 2)

1.4.28.1 Parameters

```
matrix-shape(matrix) -> array
```

1.4.29 matrix-solve

Solve `A x = b` exactly.

```
#let A = matrix($mat(2, 1; 1, -1)$)
#let b = vec((5, 1))
#to-typst(matrix-solve(A, b))
```

$$\begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

1.4.29.1 Parameters

```
matrix-solve(
  A,
  b
) -> bytes
```

1.4.30 matrix-solve-any

Solve `A x = b` exactly, allowing any solution for underdetermined systems.

```
#let A = matrix($mat(2, 1; 1, -1)$)
#let b = vec((5, 1))
#to-typst(matrix-solve-any(A, b))
```

$$\begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

1.4.30.1 Parameters

```
matrix-solve-any(
  A,
  b
) -> bytes
```

1.4.31 matrix-sub

Subtract two matrix payloads.

```
#to-typst(matrix-sub(matrix(((1, 2), (3,
4))), identity(2)))
```

$$\begin{pmatrix} 0 & 2 \\ 3 & 3 \end{pmatrix}$$

1.4.31.1 Parameters

```
matrix-sub(
  lhs,
  rhs
) -> bytes
```

1.4.32 mul

Multiply one or more expression payloads or Typst values.

```
#to-typst(mul(2, "x", "y"))
```

$$2xy$$

1.4.32.1 Parameters

```
mul(..factors) -> bytes
```

1.4.33 neg

Negate an expression payload.

```
#to-typst(neg("x"))
```

$$-x$$

1.4.33.1 Parameters

```
neg(expr) -> bytes
```

1.4.34 nsolve

Numerically solve a univariate expression near `init`.

```
#let x = var("x")
#repr(nsolve(math($x^2 - 2$), x, 1.0))
```

```
1.4142135623746899
```

1.4.34.1 Parameters

```
nsolve(
  expr,
  var,
  init,
  prec,
  max-iterations
) -> float
```

1.4.35 nsolve-system

Numerically solve a system of expressions near `init`.

```
#let x = var("x")
#let y = var("y")
#repr(nsolve-system((math($x^2 + y - 3$),
math($x - y$)), (x, y), (1.0, 1.0)))
```

```
(1.3027756377319948, 1.3027756377319948)
```

1.4.35.1 Parameters

```
nsolve-system(
  system,
  variables,
  init,
  prec,
  max-iterations
) -> array
```

1.4.36 pow

Raise `base` to `exp`.

```
#to-typst(pow("x", 3))
```

$$x^3$$

1.4.36.1 Parameters

```
pow(
  base,
  exp
) -> bytes
```

1.4.37 primitive-part

Compute the primitive part of a matrix payload.

```
#let x = var("x")
#let P = matrix(((mul(2, x), mul(4, x)),
(mul(6, x), mul(8, x))))
#to-typst(primitive-part(P))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

1.4.37.1 Parameters

```
primitive-part(matrix) -> bytes
```

1.4.38 replace

Replace subexpressions matching `pattern` with `rhs`.

```
#to-typst(replace(math($f(x, y)$),
math($f("a_", "b_")$), math($g("b_",
"a_")$)))
```

$$g(y, x)$$

1.4.38.1 Parameters

```
replace(
  expr,
  pattern,
  rhs,
  repeat,
  once,
  bottom-up,
  nested,
  non-greedy-wildcards,
  min-level,
  max-level,
  level-range,
  level-is-tree-depth,
  partial,
  allow-new-wildcards-on-rhs,
  rhs-cache-size
) -> bytes
```

1.4.39 replace-multiple

Apply multiple replacement rules in one pass.

```
#let r1 = rule(math($f("a_")$),
math($h("a_")$))
#let r2 = rule(var("x"), var("z"))
```

```
#to-typst(replace-multiple(math($f(x) + x$),
(r1, r2)))
```

$z + h(x)$

1.4.39.1 Parameters

```
replace-multiple(
  expr,
  rules,
  repeat,
  once,
  bottom-up,
  nested
) -> bytes
```

1.4.40 replace-wildcards

Replace wildcard placeholders inside a pattern.

```
#to-typst(replace-wildcards(math($k("a_")$),
((wild("a"), math($x + 1$)),)))
```

$k(1 + x)$

1.4.40.1 Parameters

```
replace-wildcards(
  pattern,
  replacements
) -> bytes
```

1.4.41 row-reduce

Row-reduce a matrix and return `(matrix: ..., rank: ...)`.

```
#let rr = row-reduce(matrix(((1, 2), (3,
4))))
rank #rr.rank: #to-typst(rr.matrix)
```

rank 2: $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

1.4.41.1 Parameters

```
row-reduce(
  matrix,
  max-col
) -> dictionary
```

1.4.42 rule

Build a reusable replacement rule for `replace-multiple`.

```
#let r = rule(math($f("a_")$),
math($g("a_")$))
#to-typst(replace-multiple(math($f(x)$),
(r,)))
```

$g(x)$

1.4.42.1 Parameters

```
rule(
  pattern,
  rhs,
  non-greedy-wildcards,
  min-level,
  max-level,
  level-range,
  level-is-tree-depth,
  partial,
  allow-new-wildcards-on-rhs,
  rhs-cache-size
) -> dictionary
```

1.4.43 series

Compute a series expansion around `expansion-point`.

```
#let sym = init(namespace: "symbolica")
#let m = sym.math
#let v = sym.var
#let ser = sym.series
#let render = sym.to-typst
#render(ser(m($cos(x)/(x + 1)$), v("x"), 0,
3))
```

$$1 - x + \frac{1}{2}x^2 - \frac{1}{2}x^3$$

1.4.43.1 Parameters

```
series(
  expr,
  var,
  expansion-point,
  depth,
  depth-denom,
  depth-is-absolute
) -> bytes
```

1.4.44 simplify

Normalize an expression payload without changing its mathematical value.

```
#to-typst(simplify(math($x + 0$)))
```

$$x$$

1.4.44.1 Parameters

```
simplify(expr) -> bytes
```

1.4.45 solve-linear

Solve a linear system exactly for `variables`.

```
#let x = var("x")
#let y = var("y")
```

```
#let sol = solve-linear((math($2 x + y - 5$), math($x - y - 1$)), (x, y))
#sol.map(to-typst).join[, ]
```

$2,\ 1$

1.4.45.1 Parameters

```
solve-linear(
  system,
  variables
) -> array
```

1.4.46 solve-system

Solve a linear or polynomial nonlinear system exactly.

Each returned row is one solution, ordered like `variables`.

1.4.46.1 Parameters

```
solve-system(
  system,
  variables
) -> array
```

1.4.47 split-col

Split a matrix at a column index.

```
#let aug = augment(identity(2), matrix(((1, 2), (3, 4))))
#let parts = split-col(aug, 2)
#to-typst(parts.at(0)) | #to-typst(parts.at(1))
```

$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \mid \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$

1.4.47.1 Parameters

```
split-col(
  matrix,
  index
) -> array
```

1.4.48 sub

Subtract two expression payloads or Typst values.

```
#to-typst(sub("x", "y"))
```

$x - y$

1.4.48.1 Parameters

```
sub(
  lhs,
  rhs
) -> bytes
```

1.4.49 to-latex

Render an atom or matrix payload as LaTeX source.

```
#raw(to-latex(math($x^2 + 1$)))
```

```
1+x^{2}
```

1.4.49.1 Parameters

```
to-latex(expr) -> str
```

1.4.50 to-typst

Render an atom or matrix payload as Typst math content.

```
#to-typst(math($x + 1$))
```

$1 + x$

1.4.50.1 Parameters

```
to-typst(
  expr,
  block
) -> content
```

1.4.51 to-typst-source

Render an atom or matrix payload as Typst math source.

```
#raw(to-typst-source(math($x + 1$)))
```

```
1+x
```

1.4.51.1 Parameters

```
to-typst-source(expr) -> str
```

1.4.52 transpose

Transpose a matrix payload.

```
#to-typst(transpose(matrix(((1, 2), (3,
4)))))
```

$\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$

1.4.52.1 Parameters

```
transpose(matrix) -> bytes
```

1.4.53 var

Construct a variable atom with an optional namespace override.

```
#to-typst(var("x"))
```

x

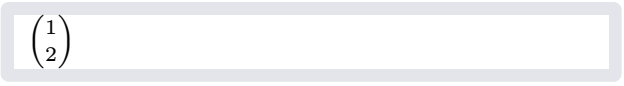
1.4.53.1 Parameters

```
var(
  name,
  namespace
) -> bytes
```

1.4.54 vec

Convert values or Typst math `vec(...)` into a column-vector matrix payload.

```
#to-typst(vec((1, 2)))
```



1.4.54.1 Parameters

```
vec(values) -> bytes
```

1.4.55 wild

Construct a wildcard symbol by appending underscore levels to `name`.

```
#raw(canonical(wild("a")))
```

```
a_
```

1.4.55.1 Parameters

```
wild(
  name,
  level,
  namespace
) -> bytes
```